

Agent memory basics

LESSON 13 · MODULE 3 — KNOWLEDGE & MEMORY

What you'll learn

- Why agents are **stateless by default** — and what that looks like in practice
- The two lines that turn memory on: a `memory=N` rolling window and a `session_id` on every `agent.run()`
- The three kinds of memory in Lyzr (conversation, Cognis, external providers) and which to reach for
- The single most common "why isn't memory working?" bug

Key concepts

Agents are stateless by default. Run two queries through the same agent and the second has no idea what happened in the first. Ask "we're a data infra team of 8 engineers" then "how many seats should we buy?" and you get a generic playbook covering every team size, because the team size from turn one didn't carry into turn two. That's what stateless looks like.

Two changes turn on memory: a window and a session. Set `memory=10` on `create_agent` for a rolling window of the last ten messages, and pass the same `session_id` to every `agent.run()`. Same string on both turns means the SDK threads them together; a different string means a different conversation with fresh context. With both in place, "how many seats?" becomes "for a data infra team of eight engineers, six seats default..." — same agent shape, two new lines of code.

The most common memory bug: a missing `session_id`. If you forget it, the SDK auto-generates a fresh one per call. Memory is technically "on," but every call lands in a new session, so nothing connects. If an agent seems to have amnesia, check the `session_id` first — before anything else.

Conversation memory covers ~80% of cases. A rolling window of the last N messages, scoped by `session_id`, living in the agent and expiring with the conversation. Reach for it whenever the agent needs to remember what the user said a couple of turns ago — chatbots, support flows, anything multi-turn.

Cognis is persistent, searchable, cross-session memory. A separate client that stores conversations long-term, scoped by owner / agent / session, and lets you search them by *meaning* later. Different session, different day, and the agent can still recall what the user told it months ago — the piece conversation memory structurally can't do. The scoping IDs keep one tenant's memory separate from another's.

TAKEAWAY

Agents are stateless by default.

TAKEAWAY

Two changes turn on memory: a window and a session.

TAKEAWAY

Conversation memory covers ~80% of cases.

TAKEAWAY

Cognis is persistent, searchable, cross-session memory.

External providers are for teams already invested in one. Mem0, AWS AgentCore, and Supermemory onboard as Studio credentials and attach to an agent's memory config. Same shape as creating any other credential. Don't introduce a new memory backend you don't need — reach for these only when your team already runs one in production.

TAKEAWAY

External providers are for teams already invested in one.

Code shown in this lesson

PYTHON

```
# Stateless by default – turn 2 can't see turn 1.
agent = studio.create_agent(
    name="onboarding-advisor", provider="gpt-4o",
    role="rollout advisor",
    goal="recommend seat counts and onboarding plans",
    instructions="Give specific, sized recommendations.",
)
agent.run("We're a data infra team of 8 engineers.")
agent.run("How many seats should we buy?") # generic – no memory of the team size
```

PYTHON

```
# Two changes: a rolling window + a session id threaded through every turn.
agent = studio.create_agent(..., memory=10) # last 10 messages

sid = "acme-data-infra"
agent.run("We're a data infra team of 8 engineers.", session_id=sid)
agent.run("How many seats should we buy?", session_id=sid)
# → "For a data infra team of 8 engineers: 6 seats default, 8 if everyone logs in, 4 minimal."
```

PYTHON

```
# Cognis – persistent semantic memory across sessions (a separate client).
cognis.add(messages, owner_id="acme", agent_id="onboarding-advisor", session_id="s1")

# Later, a different session / different wording still finds it by meaning:
hits = cognis.search("compliance evidence", owner_id="acme", agent_id="onboarding-advisor")
print(hits[0].score, hits[0].text) # 1.0 "...needs SOC 2 evidence quarterly, we use Vanta..."
```

PYTHON

```
# External providers (Mem0, AWS AgentCore, Supermemory) – onboard as a Studio credential.
cred = studio.create_memory_credential(provider="mem0", config={...})
agent = studio.create_agent(..., memory_config=cred)
```

TRY THIS

- 1** Build an agent with **no** `memory` and **no** `session_id`. Run a two-turn exchange where turn two depends on turn one. Then add `memory=10` and a shared `session_id` and rerun — diff the two answers.
- 2** Reproduce the classic bug: keep `memory=10` but *omit* `session_id`. Confirm the agent still has amnesia, then fix it by threading one `session_id`.

3

With Cognis, store a fact in session A, then search for it from session B using *different wording*. Confirm it comes back ranked by meaning, not exact match.